

HYDRAFOX

LEAD ENGINE v3.0

Claude Code — Complete Build Specification

Production-grade lead generation engine. Finds, enriches, scores, and converts \$1500+ clients — automatically. v3.0 adds a token-efficient AI pipeline (60-80% cost reduction) and a self-improvement loop that rewrites its own scoring weights and prompt strategies based on real conversion outcomes.

System	HydraFox Lead Engine
Version	3.0 — AI Optimized + Self-Improving
Target Output	20-50 Qualified Leads/Day
Min Client Value	\$1,500 USD
Core Stack	Node.js · TypeScript · Express · MongoDB · Next.js · BullMQ · Redis
AI Layer	Claude API — Tiered model strategy (Sonnet + Haiku)
New in v3	Token gate · Batch calling · Self-improving scores · Prompt A/B engine
Document	Feed to Claude Code CLI phase by phase — see Section 13

BRUTAL ANALYSIS — What the Original Blueprint Gets Wrong

These are not nitpicks. They are the exact reasons most lead gen systems die in production within 2 weeks.

PROBLEM 01	No Anti-Bot Strategy Playwright is mentioned but zero proxy rotation, fingerprint evasion, or delay randomization. Google Maps will block you within hours. This is #1 cause of scraper death.
PROBLEM 02	Naive Lead Scoring Hardcoded +20/+25 integers on booleans is amateur. No industry-value weighting, no negative signals, no confidence score. A Tier 1 salon scores the same as a Tier 3 law firm.
PROBLEM 03	No Deduplication Logic Running daily scraping jobs on the same city/industry stores the same business 15x per week. No fingerprint hashing, no domain-level dedup, no merge strategy.
PROBLEM 04	No Deliverability Architecture 'SMTP or API' is not a plan. No SPF/DKIM/DMARC setup, no sending domain warm-up, no daily send limits. Emails land in spam on day 1.
PROBLEM 05	No Error Handling or Retry BullMQ is listed but zero dead-letter queues, no partial-failure handling, no job timeout detection. ~30% of scraping jobs fail on first attempt in production.
PROBLEM 06	Vague AI Prompt System Input: HTML. Output: pain/pitch. No JSON schema enforcement, no fallback for bad outputs, no token cost control. At 50 leads/day with naive prompts = \$200+/month wasted.
PROBLEM 07	No Token Efficiency One fat AI call per lead regardless of value. No decision gate, no batching, no caching, no model tiering. 70% of those calls are unnecessary and wasteful.
PROBLEM 08	No Self-Improvement Loop Static scoring weights. Static prompts. Same failures forever. No mechanism to learn from conversion outcomes and improve automatically.

PROBLEM 09**No Data Validation**

Raw scraped data goes straight into MongoDB. No email format validation, no URL cleaning, no empty-field handling. Garbage in, garbage out.

PROBLEM 10**Zero Auth Strategy**

Dashboard has no JWT, no session management, no route protection. Not acceptable even for internal tools.

REVISED SYSTEM ARCHITECTURE

The corrected architecture adds all missing production layers. Claude Code must implement ALL of these — not just the happy path.

Layer 0	Config & Secrets	dotenv-safe, environment-based config, never hardcode keys
Layer 1	Data Ingestion	Playwright scrapers with proxy rotation + stealth fingerprinting
Layer 2	Validation Gate	Input sanitization, dedup fingerprint check, format validation
Layer 3	Enrichment Engine	Web crawl + tech detection + performance analysis + retry logic
Layer 4	AI Decision Gate	Skip AI when rules suffice (saves 70% of AI calls)
Layer 5	AI Enrichment	Tiered model calls — Sonnet for HOT, Haiku for WARM, skip for COLD
Layer 6	Scoring Engine	Weighted multi-factor model loaded from DB (dynamic weights)
Layer 7	Storage Layer	MongoDB with indexes, schema validation, TTL, dedup enforcement
Layer 8	Job Queue	BullMQ with dead-letter queues, retry limits, priority lanes
Layer 9	Cost Guard	Daily AI budget cap — halts queues if limit hit
Layer 10	API Layer	Express + JWT auth, rate limiting, Zod validation
Layer 11	Dashboard UI	Next.js SSR, protected routes, real-time job monitor
Layer 12	Outreach Engine	Email deliverability stack + LinkedIn session manager
Layer 13	Feedback Worker	Nightly job: reads outcomes, reweights scoring factors
Layer 14	Prompt Optimizer	Weekly job: rewrites underperforming prompts via AI + A/B tests
Layer 15	Problem Logger	Every scrape fail, AI fail, data issue → auto rule update

FULL TECH STACK SPECIFICATION

■ Claude Code must install and configure ALL of these. Do not substitute without explicit instruction.

Backend Core

Runtime	Node.js 20 LTS
Framework	Express.js 4.x
Language	TypeScript (strict mode)
Process Manager	PM2 (prod) / nodemon (dev)

Queue & Cache

Queue	BullMQ 5.x
Queue Backend	Redis 7.x via ioredis
Queue UI	Bull Board (admin panel)

Database

Primary	MongoDB 7.x via Mongoose
ODM	Mongoose with schema validation
Indexes	Compound on: domain, score, status, industry, created_at

Scraping

Browser	Playwright + playwright-extra stealth plugin
HTTP Client	got-scraping (fingerprint-aware, no Playwright needed for simple pages)
Proxy	Rotating residential proxies — BrightData SDK or custom rotation middleware
Rate Limiter	p-limit + token bucket per domain

Enrichment

Tech Detection	Wappalyzer (self-hosted npm package)
Performance	Lighthouse Node API (not CLI)
HTML Parsing	cheerio
DNS/WHOIS	node-dns + whois-json

AI Layer

Primary Model	claude-sonnet-4-20250514 — HOT leads + prompt optimization
Batch Model	claude-haiku-4-5-20251001 — WARM leads + batch analysis
Cost Control	guardedAICall() wrapper — daily budget cap + per-call ledger
Caching	Redis L1 (domain, 30 days) + MongoDB L2 (patterns, 7 days)
Self-Improvement	Feedback worker + Prompt optimizer (see Sections 07-08)

Frontend

Framework	Next.js 14 App Router
Styling	Tailwind CSS + shadcn/ui
State	Zustand (global) + React Query (server state)
Tables	TanStack Table v8
Charts	Recharts

Auth & Security

Auth	NextAuth.js with Credentials provider
Tokens	JWT + refresh token rotation
Rate Limiting	express-rate-limit on all API routes
Validation	Zod backend + React Hook Form + Zod frontend
Secrets	dotenv-safe — validates all required vars at startup

Email Outreach

Sending	Resend API (primary) or Nodemailer + SMTP
Deliverability	Custom domain + SPF + DKIM + DMARC — mandatory before first send
Tracking	Open/click via Resend webhooks

DevOps

Logging	Winston + daily rotate file
Errors	Sentry free tier
Deployment	Docker Compose dev — single VPS prod (DigitalOcean/Hetzner 4GB RAM min)

DATABASE SCHEMA — Complete Mongoose Models

All collections must use Mongoose schema validation. These are the authoritative field definitions. Claude Code generates models exactly matching these.

Collection: leads

<code>_id</code>	<code>ObjectId</code>	Auto
<code>businessName</code>	<code>String</code>	Required, trimmed
<code>domain</code>	<code>String</code>	Required, unique, lowercase — primary dedup key
<code>website</code>	<code>String</code>	Full URL with protocol
<code>industry</code>	<code>String</code>	Enum — see industry list
<code>industryTier</code>	<code>Number</code>	1=low 2=mid 3=high-ticket
<code>location.city</code>	<code>String</code>	
<code>location.country</code>	<code>String</code>	ISO alpha-2
<code>source</code>	<code>String</code>	Enum: gmaps linkedin instagram manual
<code>sourceId</code>	<code>String</code>	Platform-specific ID for dedup
<code>fingerprint</code>	<code>String</code>	SHA256(domain+businessName) — checked before every write
<code>score</code>	<code>Number</code>	0-100, computed by scoring engine
<code>scoreBreakdown</code>	<code>Object</code>	Per-factor scores + weights used
<code>scoringConfigVersion</code>	<code>Number</code>	Which weight config version was used
<code>confidenceScore</code>	<code>Number</code>	0-1.0 — how complete the enrichment data is
<code>leadType</code>	<code>String</code>	Enum: website automation software hybrid
<code>priority</code>	<code>String</code>	Enum: hot warm cold — derived from score
<code>status</code>	<code>String</code>	Enum: new enriched reviewed contacted won lost archived
<code>tags</code>	<code>[String]</code>	Custom labels
<code>createdAt</code>	<code>Date</code>	Auto, indexed
<code>enrichedAt</code>	<code>Date</code>	Null until enrichment completes

Collection: enrichments

<code>leadId</code>	<code>ObjectId</code>	Ref: leads — indexed
<code>techStack</code>	<code>[String]</code>	Detected tools max 10 items
<code>cms</code>	<code>String</code>	WordPress Shopify Wix Squarespace custom unknown
<code>performanceScore</code>	<code>Number</code>	Lighthouse 0-100
<code>seoScore</code>	<code>Number</code>	Lighthouse SEO 0-100

siteQualityScore	Number	Composite 0-100
mobileFriendly	Boolean	
hasSSL	Boolean	
hasChatbot	Boolean	
hasBookingSystem	Boolean	
hasEcommerce	Boolean	
automationLevel	String	Enum: none basic moderate advanced
automationSignals	[String]	Detected automation tools
socialActivity	String	Enum: active inactive unknown
detectedPains	[String]	AI-identified problems — max 5
primaryPain	String	Single most critical problem
pitchAngle	String	10-word suggested pitch hook from AI
aiAnalysisCacheKey	String	Redis key used — for cache invalidation
rawHtmlSnapshot	String	First 5000 chars stripped HTML — NOT full HTML
siteStatus	String	Enum: live unreachable redirect error
enrichedAt	Date	

Collection: contacts

leadId	ObjectId	Ref: leads — indexed
name	String	
role	String	Owner Manager Director etc.
email	String	Validated format, lowercase — indexed
phone	String	E.164 format
linkedin	String	Profile URL
confidence	String	Enum: verified guessed scraped

Collection: outreachLogs

leadId	ObjectId	Ref: leads — indexed
contactId	ObjectId	Ref: contacts
channel	String	Enum: email linkedin whatsapp instagram
promptTemplateId	ObjectId	Ref: promptTemplates — tracks which prompt was used
messageSent	String	Actual message text
sentAt	Date	
status	String	Enum: pending sent opened replied bounced failed ignored
openedAt	Date	
repliedAt	Date	
response	String	Captured reply text

followUpDate	Date	Next follow-up datetime
---------------------	-------------	-------------------------

Collection: outcomeLogs [NEW — feeds self-improvement]

leadId	ObjectId	Ref: leads
predictedScore	Number	Score at time of outreach
predictedPriority	String	What we predicted: hot warm cold
actualOutcome	String	Enum: replied interested converted ignored bounced objected
replyQuality	String	Enum: positive neutral negative none
dealValue	Number	USD if converted
daysToReply	Number	null if no reply
channelUsed	String	Which channel got the outcome
pitchAngle	String	Pain we pitched — used to score pitch strategies
scoreFactors	Object	Full scoreBreakdown snapshot at outreach time
industryTier	Number	Copied from lead for fast aggregation
promptTemplateId	ObjectId	Ref: promptTemplates — which prompt generated this outreach
loggedAt	Date	

Collection: promptTemplates [NEW — self-improving prompts]

name	String	e.g. 'outreach_law_firm', 'analysis_ecommerce'
type	String	Enum: analysis outreach_email outreach_linkedin batch_analysis
version	Number	Auto-incremented per name
template	String	Prompt text with {placeholder} variables
targetIndustry	String	null = all industries
targetService	String	null = all services
status	String	Enum: active testing retired
timesUsed	Number	
replyRate	Number	0.0-1.0 rolling average — primary performance signal
conversionRate	Number	0.0-1.0
parentVersion	Number	Which version this was generated from
generatedBy	String	Enum: system ai_optimizer
retiredAt	Date	null if still active

Collection: scoringConfigs [NEW — dynamic weights]

version	Number	Auto-incremented — scoring engine loads latest active
active	Boolean	Only one active at a time
weights	Object	Factor name → point value map (replaces hardcoded values)
industryTiers	Object	Industry name → tier number map
generatedBy	String	Enum: manual feedback_worker

changeLog	[Object]	Array of {factor, oldWeight, newWeight, reason}
createdAt	Date	

Collection: selfImproveLogs [NEW — audit trail]

type	String	Enum: weight_change prompt_update scrape_rule ai_rule outcome_pattern
severity	String	Enum: info warning critical
problem	String	What triggered this log
actionTaken	String	What the system automatically did
ruleUpdated	Boolean	Whether a config was changed
ruleDetails	Object	old value, new value, field changed
outcomesSampled	Number	How many outcomes were analysed
timestamp	Date	

Collection: costLedger [NEW — AI spend tracking]

model	String	claude-sonnet-4-20250514 claude-haiku-4-5-20251001
leadId	ObjectId	Ref: leads — nullable for batch calls
purpose	String	Enum: lead_analysis outreach_gen batch_analysis prompt_optimization
inputTokens	Number	
outputTokens	Number	
costUSD	Number	Calculated: tokens * model rate
cacheHit	Boolean	Was this served from cache (cost = \$0)?
timestamp	Date	Indexed for daily aggregation

LEAD SCORING ENGINE — Dynamic Weighted Algorithm

Scoring weights are NOT hardcoded. They are loaded from the active scoringConfigs document in MongoDB (Redis-cached for 5 minutes). The feedback worker updates them nightly based on real conversion data.

→ scoring engine = pure function: EnrichedLead → ScoreResult. No DB writes, no AI calls. Must be deterministic and run in < 10ms.

Industry Tier Multiplier

Tier 3 ×1.3	Law firms, medical clinics, real estate agencies, financial advisors, recruiting firms, enterprise software, luxury services
Tier 2 ×1.0	Gyms, dental clinics, e-commerce, marketing agencies, restaurant chains, SaaS startups, trade businesses
Tier 1 ×0.5	Solo freelancers, small cafes, local NGOs, basic retail — deprioritize

Default Scoring Factors (stored in scoringConfigs — updated by feedback worker)

DEFAULT PTS	FACTOR	CONDITION
+25	No automation tools	automationLevel === 'none'
+20	Outdated/broken website	performanceScore < 50 OR siteQualityScore < 40
+20	No SSL certificate	hasSSL === false
+15	Mobile unfriendly	mobileFriendly === false
+15	Active on social	socialActivity === 'active'
+15	No booking system	hasBookingSystem === false AND industry needs it
+10	Old CMS	cms in ['wix','squarespace'] OR outdated WP
+10	No chatbot (Tier 2+)	hasChatbot === false AND industryTier >= 2
+5	No contact form	hasContactForm === false
-15	Advanced automation stack	automationLevel === 'advanced'
-10	High-perf site 90+	performanceScore >= 90
-20	Tier 1 industry	industryTier === 1
-999	Duplicate domain	fingerprint exists in DB — BLOCK

Score → Priority Map

80 - 100	HOT	Outreach within 24h
55 - 79	WARM	Outreach within 72h
30 - 54	COLD	Batch outreach weekly
0 - 29	SKIP	Do not contact

AI TOKEN OPTIMIZATION ENGINE

60-80% cost reduction. 3x faster. Zero quality loss.

The naive system calls Claude once per lead — always — with raw HTML. At 50 leads/day that burns \$3-5/day in unnecessary API calls. The v3 AI engine uses a 4-layer optimization stack: gate, tier, compress, cache.

6.1 — AI Decision Gate (Skip AI on 70% of leads)

Before EVERY AI call, this gate runs. It decides whether AI is actually needed. Most leads can be fully scored by rules alone. AI is only called when it adds value.

```
// src/ai/gate.ts
export type GateVerdict = "cache_hit" | "skip_dead" | "skip_low_value"
                        | "skip_rules_sufficient" | "call_sonnet" | "call_haiku";

export async function aiGate(lead: EnrichedLead): Promise {
  // 1. Cache check – same domain seen before (30-day TTL)
  const cacheKey = `ai:analysis:${lead.fingerprint}`;
  if (await redis.exists(cacheKey)) return "cache_hit";

  // 2. Dead site – nothing to analyze
  if (lead.enrichment.siteStatus === "unreachable") return "skip_dead";

  // 3. Tier 1 + low score – not worth tokens
  if (lead.industryTier === 1 && lead.score < 35) return "skip_low_value";

  // 4. Rule coverage is high enough – AI adds nothing
  const rulesCoverage = countDefinedFactors(lead.scoreBreakdown) / TOTAL_FACTORS;
  if (rulesCoverage >= 0.85 && lead.score > 75) return "skip_rules_sufficient";
  if (rulesCoverage >= 0.85 && lead.score < 35) return "skip_low_value";

  // 5. Route to correct model tier
  if (lead.score >= 80) return "call_sonnet"; // HOT – full analysis
  return "call_haiku"; // WARM – pain detection only
}
// Expected: ~30% of leads reach AI. 70% handled by rules + cache.
```

✓ Result: Daily AI spend drops from ~\$4.50 → ~\$0.90 at 50 leads/day. No loss in lead quality — COLD leads don't need AI analysis anyway.

6.2 — Tiered Model Strategy

Lead Type	Model	Task	Token Budget
HOT lead (80+)	claude-sonnet-4-20250514	Full analysis + personalized pitch	~800 out tokens
WARM lead (55-79)	claude-haiku-4-5-20251001	Pain detection only — no pitch yet	~300 out tokens
COLD lead (<55)	SKIP — rules only	Queue for batch review	\$0

Batch outreach	claude-haiku-4-5-20251001	6 messages in 1 call	~1200 out tokens
Prompt optimizer	claude-sonnet-4-20250514	Weekly run — rewrites bad prompts	~2000 out tokens
Feedback analysis	claude-sonnet-4-20250514	Weekly — synthesizes outcomes	~1500 out tokens

6.3 — HTML Compression (86% input token reduction)

Never send raw HTML to Claude. Pre-extract all signals into a structured compressed object. Raw HTML = ~2500 tokens. Compressed context = ~350 tokens.

```
// src/ai/compressor.ts
export function compressForAI(lead: EnrichedLead): CompressedContext {
  return {
    biz: {
      name: lead.businessName, industry: lead.industry,
      tier: lead.industryTier, location: `${lead.location.city}, ${lead.location.country}`,
    },
    tech: {
      cms: lead.enrichment.cms,
      stack: lead.enrichment.techStack.slice(0, 5), // cap at 5 – rest is noise
      perf: lead.enrichment.performanceScore,
      mobile: lead.enrichment.mobileFriendly,
    },
    flags: { // Pre-computed booleans – no HTML needed
      noAutomation: lead.enrichment.automationLevel === "none",
      noBooking: !lead.enrichment.hasBookingSystem,
      noChatbot: !lead.enrichment.hasChatbot,
      poorUX: lead.enrichment.siteQualityScore < 45,
      inactive: lead.enrichment.socialActivity === "inactive",
    },
    // Strip ALL HTML tags + scripts + styles. Keep only visible text. Cap at 400 chars.
    snippet: stripHtml(lead.enrichment.rawHtmlSnapshot).slice(0, 400),
  };
}

function stripHtml(html: string): string {
  return html
    .replace(/<\/?g, "")
    .replace(/<\/?g, "")
    .replace(/<[^\>]+>/g, " ")
    .replace(/\\s+/g, " ").trim();
}
```

6.4 — Batch Calling (6 WARM leads per API call)

Instead of 6 separate API calls for WARM leads, bundle them into 1. Claude handles multi-lead JSON output reliably when the schema is enforced.

```
SYSTEM: Analyze multiple businesses. Return ONLY valid JSON. Zero text outside the JSON object.

USER: Analyze these businesses and return exactly this structure:
{
  "analyses": [
    {
      "id": "{lead_id}",
      "primaryPain": "1 sentence – specific to this business",
      "pains": ["string", "string"], // max 3, no generic filler
    }
  ]
}
```

```
    "serviceMatch": "website|automation|software|hybrid",
    "confidence": 0.0-1.0,                // below 0.5 = not enough data
    "pitchAngle": "10 words max — hook referencing their actual problem"
  }
]
}
```

Businesses: {LEADS_JSON}

Rules: Be specific per business. pitchAngle must reference a real detected signal.
Confidence < 0.5 is valid — don't guess. Response must be parseable JSON.

✓ 6 WARM leads in 1 API call = 83% fewer API calls for that segment. Cost per WARM lead: \$0.004 → \$0.0007.

6.5 — Multi-Layer Cache Architecture

L1 — Domain Cache	Redis	30 days	Full AI analysis by domain fingerprint. Re-scraping same business = \$0 AI cost.
L2 — Industry Pattern Cache	Redis	7 days	Common pain combos per industry. Law firm #47 matches law firm #12 pattern → reuse.
L3 — Outreach Template Cache	MongoDB	24 hours	Outreach messages cached per pain+service combo. Skip regenerating identical pitches.
L4 — Negative Cache	Redis	90 days	Domains where outreach was rejected. Never re-analyze or re-pitch these.

6.6 — Cost Guard Worker (Hard Daily Budget Enforcement)

```
// src/workers/costGuard.worker.ts
// Wrap EVERY Claude API call with this — no exceptions
const DAILY_BUDGET_USD = 5.00;           // Adjust in /settings
const PRICING = {
  "claude-sonnet-4-20250514": { in: 0.000003, out: 0.000015 },
  "claude-haiku-4-5-20251001": { in: 0.00000025, out: 0.00000125 },
};

export async function guardedAICall(params: AICallParams): Promise {
  const todayKey = `cost:${new Date().toISOString().slice(0,10)}`;
  const spent = parseFloat(await redis.get(todayKey) || "0");

  if (spent >= DAILY_BUDGET_USD) {
    logger.warn(`Daily AI budget cap hit (${DAILY_BUDGET_USD}). Halting enrichment queue.`);
    await bullMQ.pauseQueue("enrichment");
    return null; // Caller must handle null — use rule-only fallback
  }

  const result = await claudeClient.messages.create(params.request);
  const cost = (result.usage.input_tokens * PRICING[params.model].in)
    + (result.usage.output_tokens * PRICING[params.model].out);
```

```
    await redis.incrbyfloat(todayKey, cost);
    await redis.expire(todayKey, 86400);
    await CostLedger.create({ ...params, ...result.usage, costUSD: cost, timestamp: new Date() })
;

    return result;
}
```

AI PROMPT SYSTEM — Dynamic + Self-Improving Templates

All prompts are stored in the promptTemplates collection — not hardcoded. The system tracks reply rates per prompt and automatically generates improved versions weekly. Claude Code must implement prompts as template functions that load from DB, not as string constants.

■ Never hardcode prompt text. Always load from promptTemplates collection. This is what makes the system self-improving.

Prompt 1 — HOT Lead Analysis (claude-sonnet)

Called for HOT leads (80+ score) only. Uses compressed context — NOT raw HTML.

SYSTEM: You are a business analyst identifying digital inefficiencies. Respond ONLY with valid JSON. No preamble, no markdown, no explanation outside the JSON.

USER: Analyze this business and return exactly this schema:

```
{
  "primaryPain": "1 sentence — the single most critical business problem you detected",
  "pains": ["string", "string", "string"],          // max 3, specific to this business
  "serviceMatch": "website|automation|software|hybrid",
  "estimatedValue": "low|mid|high",                // expected deal value
  "confidence": "0.0-1.0",
  "pitchAngle": "10 words max — hook referencing their actual detected problem",
  "summary": "2 sentences max — what this business is and why they're a lead"
}
```

```
Business: {businessName} | Industry: {industry} | Tier: {industryTier}
Location: {location}
Tech: {cms} | Stack: {techStack} | Performance: {performanceScore}/100
Flags: noAutomation={noAutomation} | noBooking={noBooking} | poorUX={poorUX}
Content snippet (150 words visible text): {snippet}
```

Prompt 2 — Outreach Generation (claude-haiku for speed)

Only called after manual review approval. Never auto-send. Human always reviews before send.

SYSTEM: You are a direct-response copywriter. Write outreach that sounds human and specific. Never mention AI. Never use generic openers. Respond ONLY with valid JSON.

USER: Write outreach for this lead and return:

```
{
  "subject": "Email subject under 50 chars — no clickbait",
  "emailBody": "3 paragraphs. Para 1: specific observation about their business. Para 2: what y
ou do. Para 3: low-friction CTA.",
  "linkedinDM": "Under 300 chars — casual, no pitch in first message",
  "openingHook": "First sentence that earns attention — references a specific problem"
}
```

```
Business: {businessName} | Contact: {contactName} ({contactRole})
```

```
Detected pain: {primaryPain}  
Service to pitch: {serviceMatch} | Pitch angle: {pitchAngle}  
Your agency: {agencyName} | Your offer: {offerSummary}
```

✓ Cost control: Cache outreach templates by pain+service combo for 24 hours. Re-scraping the same domain within 30 days = load AI analysis from Redis cache, cost \$0.

SELF-IMPROVEMENT ENGINE

The system learns from every outcome and improves itself automatically.

This is the moat. Most lead gen tools are static — same prompts, same weights, same failures forever. HydraFox v3 treats every outreach outcome as training signal. Scoring weights are retuned nightly. Prompts are rewritten weekly. Scraping rules update when blocks occur. The system compounds.

8.1 — Outcome Logging (Mandatory — feeds everything)

Every lead that gets outreach sent MUST have its outcome logged. This is raw training data. The UI must make this frictionless — one click max.

■ If outcome logging takes more than 3 seconds, operators stop doing it and the entire self-improvement loop dies. Build the UI for speed — keyboard shortcuts, one-click status update.

8.2 — Feedback Worker (Nightly — 2am via BullMQ cron)

```
// src/workers/feedback.worker.ts  BullMQ cron: "0 2 * * *"
export async function runFeedbackAnalysis(): Promise {
  const outcomes = await OutcomeLog.find({
    loggedAt: { $gte: subDays(new Date(), 7) }
  });
  if (outcomes.length < 10) return; // Not enough data yet

  // 1. Compute which scoring factors actually correlated with replies
  const factorPerf = computeFactorCorrelations(outcomes);

  // 2. Compute per-industry conversion rates
  const industryStats = computeIndustryStats(outcomes);

  // 3. Compute which pitch angles got replies
  const pitchStats = computePitchStats(outcomes);

  // 4. Auto-retune weights for high-confidence factors
  const retuneResult = await retuneScoringWeights(factorPerf);

  // 5. Flag underperforming industries for tier demotion review
  const tierFlags = flagUnderperformingIndustries(industryStats);

  // 6. Log everything for audit trail
  await SelfImproveLog.create({
    type: "weight_change", runAt: new Date(),
    outcomesSampled: outcomes.length,
    actionTaken: `Auto-retuned ${retuneResult.autoApplied.length} weights`,
    ruleDetails: retuneResult.autoApplied,
  });
}

function computeFactorCorrelations(outcomes: OutcomeLog[]): FactorPerformance {
  const FACTORS = ["noAutomation", "outdatedWebsite", "noSSL", "mobileUnfriendly",
    "noBooking", "noChatbot", "oldCMS", "poorUX", "activeOnSocial"];
}
```

```

const result: FactorPerformance = {};
for (const factor of FACTORS) {
  const withFactor = outcomes.filter(o => o.scoreFactors[factor] === true);
  const withoutFactor = outcomes.filter(o => o.scoreFactors[factor] === false);
  const rateWith = replyRate(withFactor);
  const rateWithout = replyRate(withoutFactor);
  result[factor] = {
    lift: withFactor.length > 0 ? (rateWith - rateWithout) / Math.max(rateWithout, 0.01) : 0,
    replyRateWith: rateWith,
    sampleSize: withFactor.length,
    confidence: withFactor.length >= 20 ? "high" : withFactor.length >= 8 ? "medium" : "low",
  };
}
return result;
}

```

8.3 — Automatic Scoring Weight Retuner

Based on correlation data, weights are adjusted automatically. Small safe adjustments apply immediately. Large changes queue for operator review in the dashboard.

```

// src/ai/optimizer/retuner.ts
const MAX_AUTO_ADJUST = 5; // Max point change per factor per run
const CONFIDENCE_REQ = "medium";

export async function retuneScoringWeights(factorPerf: FactorPerformance) {
  const current = await ScoringConfig.findOne({ active: true });
  const weights = { ...current.weights };
  const autoApplied: WeightChange[] = [];
  const requiresReview: WeightChange[] = [];

  for (const [factor, perf] of Object.entries(factorPerf)) {
    if (perf.confidence === "low") continue;
    const current_w = weights[factor] || 0;
    let delta = 0;
    if (perf.lift > 0.30 && current_w < 30) delta = +Math.min(perf.lift * 10, MAX_AUTO_ADJUST);
    if (perf.lift < -0.20 && current_w > 5) delta = -Math.min(Math.abs(perf.lift * 10), MAX_AUTO_ADJUST);
    if (Math.abs(delta) < 1) continue;

    const change = { factor, oldWeight: current_w, newWeight: Math.round(current_w + delta),
      lift: perf.lift, confidence: perf.confidence };

    if (Math.abs(delta) >= MAX_AUTO_ADJUST || perf.confidence !== "high") {
      requiresReview.push(change);
    } else {
      autoApplied.push(change);
      weights[factor] = change.newWeight;
    }
  }

  if (autoApplied.length) {
    // Create new config version — old one stays for audit trail
    await ScoringConfig.create({
      active: true, weights, version: current.version + 1,
      generatedBy: "feedback_worker", changeLog: autoApplied, createdAt: new Date()
    });
  }
}

```

```

    await ScoringConfig.updateOne({ _id: current._id }, { active: false });
    await redis.del("scoring:config:active"); // Bust cache — engine picks up new weights
  }
  return { autoApplied, requiresReview };
}

```

8.4 — Prompt Optimizer Worker (Weekly — 3am Sunday)

Identifies underperforming prompts (< 15% reply rate, 50+ uses) and uses Claude to generate improved versions. Automatically A/B tests them — 20% traffic to new version.

```

// src/workers/optimizer.worker.ts  BullMQ cron: "0 3 * * 0"
export async function runPromptOptimizer(): Promise {
  const underperforming = await PromptTemplate.find({
    status: "active", timesUsed: { $gte: 50 }, replyRate: { $lt: 0.15 }
  });

  for (const prompt of underperforming) {
    const failures = await OutreachLog.find({ promptTemplateId: prompt._id,
      status: { $in: ["ignored", "bounced"] } }).limit(10);
    const successes = await OutreachLog.find({ promptTemplateId: prompt._id,
      status: "replied" }).limit(5);

    // Ask Claude to rewrite the prompt based on what worked vs what didn't
    const improved = await guardedAICall({
      model: "claude-sonnet-4-20250514", purpose: "prompt_optimization",
      request: { max_tokens: 1200, messages: [{ role: "user",
        content: OPTIMIZER_META_PROMPT
          .replace("{CURRENT}", prompt.template)
          .replace("{REPLY_RATE}", prompt.replyRate.toFixed(2))
          .replace("{FAILURES}", failures.map(f => f.messageSent).join("
---
").slice(0,1500))
          .replace("{SUCCESES}", successes.map(s => s.messageSent).join("
---
").slice(0,800))
          .replace("{INDUSTRY}", prompt.targetIndustry || "general")
        }}}
      });

    if (!improved) continue;

    // Save as "testing" — not active. A/B test routes 20% traffic to it.
    await PromptTemplate.create({
      ...prompt.toObject(), _id: undefined,
      template: improved.content[0].text,
      version: prompt.version + 1, status: "testing",
      timesUsed: 0, replyRate: 0, parentVersion: prompt.version,
      generatedBy: "ai_optimizer", createdAt: new Date(),
    });
  }
  await graduateTestedPrompts(); // Promote winners, retire losers
}

const OPTIMIZER_META_PROMPT = `
You are a cold outreach conversion expert. Rewrite this underperforming prompt.
Current reply rate: {REPLY_RATE} Industry: {INDUSTRY}

```

CURRENT PROMPT:

{CURRENT}

MESSAGES THAT GOT IGNORED (learn what NOT to do):

{FAILURES}

MESSAGES THAT GOT A REPLY (learn from these):

{SUCCESES}

Rules: Keep all {placeholder} variables. Be more specific, more human, 20% shorter.

The hook must reference a concrete detected problem – not a generic benefit.

Return ONLY the improved prompt text. No explanation.`;

8.5 — Prompt A/B Testing + Auto-Graduation

```
// src/ai/prompt.manager.ts
export async function selectPrompt(type: string, industry?: string): Promise {
  const candidates = await PromptTemplate.find({
    type, status: { $in: ["active","testing"] },
    $or: [{ targetIndustry: null }, { targetIndustry: industry }],
  });
  const testing = candidates.filter(p => p.status === "testing");
  const active = candidates.filter(p => p.status === "active")
    .sort((a,b) => b.replyRate - a.replyRate);

  // 20% traffic to testing variant (if one exists)
  return (testing.length > 0 && Math.random() < 0.20) ? testing[0] : active[0];
}

async function graduateTestedPrompts(): Promise {
  const tested = await PromptTemplate.find({ status:"testing", timesUsed:{ $gte:30 } });
  for (const t of tested) {
    const parent = await PromptTemplate.findOne({ name: t.name, version: t.parentVersion });
    if (!parent) continue;
    if (t.replyRate > parent.replyRate * 1.15) { // 15%+ improvement → graduate
      await PromptTemplate.updateOne({ _id: t._id }, { status: "active" });
      await PromptTemplate.updateOne({ _id: parent._id }, { status: "retired", retiredAt: new Date() });
      await SelfImproveLog.create({ type:"prompt_update",
        actionTaken:`Graduated ${t.name} v${t.version} (${(t.replyRate*100).toFixed(1)}% vs ${((parent.replyRate*100).toFixed(1)}%)` });
    } else if (t.timesUsed >= 100) { // Tested enough, didn't improve → retire
      await PromptTemplate.updateOne({ _id: t._id }, { status: "retired" });
    }
  }
}
```

OPERATIONAL PROBLEM LEARNING

Every scraping block, AI failure, and data issue feeds back into automatic rule updates.

The bot doesn't just learn from conversions. It learns from its own operational failures. Blocked by a proxy? Auto-increase delays for that city. AI returning bad JSON repeatedly? Auto-switch that lead type to rule-only. Certain industries always have dead sites? Auto-reduce their tier weight.

PROBLEM	Google Maps blocks IP 3x in same city within 1 hour
AUTO-FIX	Increase minimum delay for that city from 800ms → 3000ms. Suspend city queue 4 hours. Log to selfImproveLogs. Alert operator.
PROBLEM	AI returns invalid JSON 2x for same lead type
AUTO-FIX	Flag lead type as 'ai-resistant'. Switch to rule-only scoring for that type. Queue failing prompts for optimizer.
PROBLEM	Enrichment timeout rate > 40% for a specific domain suffix
AUTO-FIX	Add domain suffix to slow-fetch list. Increase timeout threshold for that TLD. Log pattern.
PROBLEM	Industry X has < 5% reply rate over 30+ leads
AUTO-FIX	Flag for industry tier review. Auto-add to requiresReview queue in selfImproveLogs. Do not auto-demote tier — human decides.
PROBLEM	Proxy pool exhaustion detected
AUTO-FIX	Halt all scraping queues immediately. Log CRITICAL. Resume only when proxies available. Never scrape without proxy.
PROBLEM	Duplicate rate > 30% in a scraping job
AUTO-FIX	Reduce scraping limit for that city/industry combination. Expand geographic search radius instead.
PROBLEM	Cost guard triggers (budget cap hit before 6pm)
AUTO-FIX	Log which leads consumed the most tokens. Automatically lower WARM lead batch size for next day. Review prompt efficiency.
PROBLEM	Lead scored 80+ but outreach gets 0 replies over 10+ sends
AUTO-FIX	Auto-flag that score factor combination as 'over-scoring'. Add to requiresReview in next feedback worker run.

SCRAPING ARCHITECTURE — Anti-Detection Requirements

■ This section is NON-NEGOTIABLE. Skip any of these and the scraper will be blocked within 48 hours.

Google Maps Scraper

- **Playwright + stealth:** Use playwright-extra with stealth plugin. Real user-agent from rotating pool of 20.
- **Human delays:** Random 800ms–3500ms between actions. Never fixed intervals.
- **Proxy rotation:** Rotate residential proxies per request. Never reuse IP for same city within 1 hour.
- **Viewport randomization:** Random size from pool of 10 real device dimensions per session.
- **Scroll simulation:** Simulate human scroll behavior when loading map results.
- **Session limits:** Max 50 extractions per browser context. Fresh context each session.
- **CAPTCHA detection:** Detect CAPTCHA pages — pause job, log proxy IP, retry in 4 hours with fresh proxy.
- **Data per business:** Name, address, phone, website, rating, review count, category, hours, Maps URL

Website Enrichment Crawler

- **Timeout:** Hard 10s timeout per page. Mark as unreachable if timeout — do NOT retry immediately.
- **HTML truncation:** Store ONLY first 5000 chars. Never full page — DB cost and AI token waste.
- **Redirect handling:** Follow up to 3 redirects. Log final URL as canonical domain.
- **Error classification:** 404 = dead site (-30 score). 5xx = retry twice then mark error. SSL error = hasSSL=false.
- **robots.txt:** Check before crawling. If disallowed — use only publicly visible metadata.
- **Concurrency:** Max 5 concurrent enrichment workers. p-limit enforced. Never exceed.

API ROUTES — Express.js Specification

All routes require JWT auth except /api/auth/*. All POST/PUT/PATCH require Zod validation. All list routes support ?page=1&limit;=25 pagination.

METHOD	ROUTE	DESCRIPTION
POST	/api/auth/login	Credentials → JWT + refresh token
POST	/api/auth/refresh	Refresh token → new JWT
POST	/api/jobs/scrape	Queue scraping job {source, location, industry, limit}
GET	/api/jobs	List jobs with status and progress
DELETE	/api/jobs/:id	Cancel queued job
GET	/api/leads	List leads — filter: score, priority, status, industry, source
GET	/api/leads/:id	Single lead with full enrichment + AI analysis
PATCH	/api/leads/:id/status	Update lead status
POST	/api/leads/:id/enrich	Manually trigger enrichment
GET	/api/leads/:id/contacts	Contacts for this lead
POST	/api/leads/:id/contacts	Add contact
POST	/api/outreach/generate	Generate outreach message via AI (uses prompt selection)
POST	/api/outreach/send	Log manual outreach send
GET	/api/outreach	List all outreach logs
PATCH	/api/outreach/:id/status	Update status: opened replied bounced
POST	/api/outcomes	Log lead outcome — feeds self-improvement loop
GET	/api/analytics/summary	Dashboard stats: totals, today pipeline, cost today
GET	/api/analytics/funnel	Lead funnel by status
GET	/api/analytics/cost	AI spend by day, model, purpose
GET	/api/analytics/prompts	Prompt performance table — reply rates per version
GET	/api/analytics/weights	Current scoring weights + change history
GET	/api/self-improve/logs	Self-improvement audit trail
GET	/api/self-improve/pending	Weight/prompt changes requiring operator review
POST	/api/self-improve/approve/:id	Approve a pending weight or prompt change

FRONTEND VIEWS — Next.js 14 Page Specifications

Main Dashboard /dashboard

- Stats bar: total leads, hot leads today, outreach sent, AI spend today, conversion rate
- 7-day lead volume chart (Recharts LineChart)
- Job status widget: active scraping jobs with live progress (SSE polling every 5s)
- Recent leads table: last 10 — name / score badge / priority / status
- Self-improvement status: last feedback run, weights changed, prompts updated
- Quick actions: Start Scrape Job, View Hot Leads

Leads Table /leads

- Full paginated TanStack Table — 25 leads/page
- Columns: Business Name, Industry, Score (colored badge), Priority, Status, Source, Date
- Filters: score range slider, priority dropdown, status multi-select, industry select
- Sort by score (default desc), date, name
- Bulk actions: export CSV, bulk status update
- Search: fuzzy search by business name or domain

Lead Detail /leads/[id]

- Business header: name, location, industry, score badge, priority badge, status selector
- Two-column: left = business details + enrichment signals, right = AI analysis + contacts
- Tech stack pills, performance score bars, automation level indicator
- Detected pains list with confidence score
- Contacts section: cards with email/LinkedIn links
- Outreach panel: pitch suggestion + Generate Message button + message history
- One-click outcome logging: Replied / Ignored / Converted / Bounced (feeds self-improvement)
- Activity timeline: all actions taken on this lead

Outreach Center /outreach

- Table: Business, Contact, Channel, Prompt Version Used, Status, Sent Date, Follow-up Date
- Quick outcome logging — one click to mark replied/ignored/converted
- Filter by channel, status, priority, date range
- Follow-up scheduler: set next touch date

Job Monitor /jobs

- Active jobs with real-time progress (SSE or 5s polling)
- Failed job detail: error message + Retry button
- Queue stats: waiting, active, completed today, failed today
- New job form: source, location, industry, limit

Self-Improvement Dashboard [NEW] /self-improve

- Scoring weight history: table of all weight changes with reason and date
- Factor performance chart: reply rate per scoring factor (bar chart)
- Pending changes: weight/prompt updates requiring operator approval
- Prompt performance table: all templates, reply rate, version, status
- AI cost breakdown: spend by day, model, purpose (line + bar charts)
- Problem log: operational issues and auto-fixes applied

Settings /settings

- API keys: Anthropic key, proxy config (all masked)
- AI budget: daily cap slider, model preference per lead tier
- Scraping config: default limits, delay ranges, proxy toggle
- Outreach identity: agency name, offer summary (injected into AI prompts)
- Self-improvement: enable/disable auto weight updates, minimum confidence required
- Industry tiers: assign/reassign industries to tiers

ERROR HANDLING & RETRY ARCHITECTURE

Every async operation must be wrapped with explicit error handling. Generic try/catch is not enough — each error type has a specific response. Implement these exactly.

■ Scraper Blocked / CAPTCHA	Pause job. Move to dead-letter queue. Log proxy IP. Trigger problem learning rule. Retry in 4 hours with fresh proxy + increased delay.
■ Website Timeout / 404 / 5xx	Mark lead as enrichment_failed. Store error code. Retry enrichment once after 24h. After 2 failures: mark dead_site, score penalty -30, deprioritize.
■ AI API Timeout / Rate Limit	Exponential backoff: 1s, 4s, 16s. After 3 failures: store partial enrichment, continue without AI, use rule-only scoring. Log cost impact.
■ AI Returns Invalid JSON	safeParseJSON in try/catch. Log raw response. Use rule-based fallback. Increment fail counter — if 2+ fails for same lead type, trigger problem learning rule.
■ MongoDB Write Failure	Retry 3 times with 500ms delay. If all fail: write to file-based fallback log. Alert operator. Halt that specific job — do NOT retry the whole batch.
■ BullMQ Job Stuck > 10 min	BullMQ stalledInterval detects it. Auto-mark failed. Move to dead-letter. Alert via logger.
■ Duplicate Lead Detected	Check fingerprint BEFORE write. If exists: update last_seen + source on existing record. Never create duplicate.
■ Proxy Pool Exhausted	HALT all scraping queues immediately. Log CRITICAL. Never scrape without proxy. Resume only when proxies restored.
■ Cost Guard Cap Hit	Pause enrichment queue. Log which leads consumed most tokens. Scale back WARM batch size for next day. Notify via dashboard alert.
■ Feedback Worker — Insufficient Data	Skip silently if < 10 outcomes. Log 'insufficient data' with outcome count. Do NOT run partial analysis.

PROJECT FOLDER STRUCTURE

■ Claude Code must create this exact monorepo. Do not flatten. Every layer is separated.

```

hydrafox/
├── apps/
│   ├── backend/
│   │   ├── src/
│   │   │   ├── api/                                # Express routers
│   │   │   │   ├── auth.router.ts
│   │   │   │   ├── leads.router.ts
│   │   │   │   ├── jobs.router.ts
│   │   │   │   ├── outreach.router.ts
│   │   │   │   ├── outcomes.router.ts    # NEW – outcome logging
│   │   │   │   ├── analytics.router.ts
│   │   │   │   ├── selfImprove.router.ts # NEW
│   │   │   │   └── workers/              # BullMQ workers
│   │   │   │       ├── scraping.worker.ts
│   │   │   │       ├── enrichment.worker.ts
│   │   │   │       ├── scoring.worker.ts
│   │   │   │       ├── feedback.worker.ts # NEW – nightly
│   │   │   │       ├── optimizer.worker.ts # NEW – weekly
│   │   │   │       └── costGuard.worker.ts # NEW – per-call wrapper
│   │   │   ├── scrapers/
│   │   │   │   ├── gmaps.scrapper.ts
│   │   │   │   ├── linkedin.scrapper.ts
│   │   │   │   └── base.scrapper.ts
│   │   │   ├── enrichment/
│   │   │   │   ├── website.enricher.ts
│   │   │   │   ├── tech.detector.ts
│   │   │   │   └── performance.analyzer.ts
│   │   │   ├── ai/
│   │   │   │   ├── gate.ts                # NEW – AI decision gate
│   │   │   │   ├── compressor.ts          # NEW – HTML compression
│   │   │   │   ├── client.ts
│   │   │   │   ├── batch.analyzer.ts      # NEW – batch WARM leads
│   │   │   │   ├── prompt.manager.ts      # UPDATED – loads from DB + A/B
│   │   │   │   ├── prompts/
│   │   │   │   │   ├── analysis.prompt.ts
│   │   │   │   │   ├── outreach.prompt.ts
│   │   │   │   │   └── optimizer.prompt.ts # NEW
│   │   │   │   └── optimizer/              # NEW – self-improvement
│   │   │   │       ├── correlations.ts      # Factor correlation engine
│   │   │   │       ├── retuner.ts          # Weight auto-retuner
│   │   │   │       └── problemLearner.ts    # Operational problem rules
│   │   │   ├── models/
│   │   │   │   ├── lead.model.ts
│   │   │   │   ├── enrichment.model.ts
│   │   │   │   ├── contact.model.ts
│   │   │   │   ├── outreach.model.ts
│   │   │   │   ├── outcomeLog.model.ts    # NEW
│   │   │   │   ├── promptTemplate.model.ts # NEW
│   │   │   │   ├── scoringConfig.model.ts # NEW
│   │   │   │   ├── selfImproveLog.model.ts # NEW
│   │   │   │   └── costLedger.model.ts     # NEW
│   │   │   ├── scoring/
│   │   │   │   ├── score.engine.ts        # UPDATED – loads weights from DB
│   │   │   │   └── industry.tiers.ts
│   │   │   └── middleware/

```

```

■ ■ ■ ■ ■ auth.middleware.ts
■ ■ ■ ■ ■ rateLimit.middleware.ts
■ ■ ■ ■ ■ validate.middleware.ts
■ ■ ■ ■ ■ lib/
■ ■ ■ ■ ■ db.ts
■ ■ ■ ■ ■ redis.ts
■ ■ ■ ■ ■ logger.ts
■ ■ ■ ■ ■ proxy.manager.ts
■ ■ ■ ■ ■ utils/
■ ■ ■ ■ ■ dedup.ts
■ ■ ■ ■ ■ fingerprint.ts
■ ■ ■ ■ ■ sanitize.ts
■ ■ ■ ■ ■ index.ts
■ ■ ■ ■ ■ frontend/
■ ■ ■ ■ ■ app/
■ ■ ■ ■ ■ (dashboard)/
■ ■ ■ ■ ■ dashboard/
■ ■ ■ ■ ■ leads/[id]/
■ ■ ■ ■ ■ outreach/
■ ■ ■ ■ ■ jobs/
■ ■ ■ ■ ■ self-improve/ # NEW
■ ■ ■ ■ ■ settings/
■ ■ ■ ■ ■ api/
■ ■ ■ ■ ■ docker-compose.yml
■ ■ ■ ■ ■ docker-compose.prod.yml
■ ■ ■ ■ ■ .env.example
```

30-DAY BUILD PLAN

✓ Feed this to Claude Code phase by phase. Complete each phase fully before starting the next.

Phase 1 Days 1–5 Foundation

- Initialize monorepo with npm workspaces + TypeScript strict mode
- Docker Compose: MongoDB, Redis, BullMQ
- Create ALL Mongoose models from Section 04 (including new v3 models)
- Express app: JWT auth, rate limiting, Zod validation middleware
- Deduplication + fingerprint utility
- Queue manager: scraping, enrichment, scoring, feedback, optimizer queues
- Winston logger + dotenv-safe config
- Cost Guard wrapper (guardedAICall) — stub Anthropic client for now

Phase 2 Days 6–10 Scraper + Enrichment

- Playwright scraper with stealth plugin + proxy rotation manager
- Google Maps scraper: city + industry → business list
- CAPTCHA detection + automatic job pause + problem learner hook
- Website enrichment crawler: Wappalyzer + Lighthouse + cheerio
- HTML compressor (compressForAI) for stripped text snapshot
- Connect all to BullMQ workers with dead-letter queues

Phase 3 Days 11–15 AI Engine + Scoring

- AI Decision Gate (aiGate function) — all 5 verdict types
- Batch analyzer for WARM leads (6 per call)
- HOT lead analysis via claude-sonnet
- Scoring engine loading weights from scoringConfigs collection
- Cost Ledger writes on every AI call
- All 4 cache layers (Redis L1/L2, MongoDB L3/L4)

Phase 4 Days 16–20 API + Dashboard Core

- All Express API routes from Section 11 with Zod validation
- Next.js 14 with NextAuth.js setup
- Dashboard, Leads Table, Lead Detail pages
- One-click outcome logging UI (max 1 click to log — critical for self-improvement)
- Job Monitor with SSE real-time progress

Phase 5 Days 21–25 **Self-Improvement Layer**

- Feedback Worker: factor correlations + weight retuner (nightly cron)
- Prompt Optimizer Worker: underperformer detection + Claude rewrite (weekly cron)
- Prompt A/B testing: 20/80 traffic split + auto-graduation
- Problem Learner: all 8 operational rules from Section 09
- Self-Improve Dashboard page + pending approvals UI
- Analytics: cost breakdown, prompt performance, weight history charts

Phase 6 Days 26–30 **Polish + Deploy**

- Outreach generation with prompt.manager (DB-loaded templates)
- Outreach Center page + follow-up scheduler
- Error boundaries on all frontend pages
- Sentry error tracking setup
- docker-compose.prod.yml + VPS deployment (DigitalOcean 4GB RAM min)
- 3-day live test: 50 leads/day — verify dedup, AI gate, cost tracking
- Seed initial promptTemplates and scoringConfigs documents

INSTRUCTIONS FOR CLAUDE CODE — How to Use This Document

- Feed this document to Claude Code CLI phase by phase — never all at once. One phase per session. Reference section numbers explicitly in your prompts.

Starter Prompt — Feed this first

You are building HydraFox Lead Engine v3.0 — a production lead generation system with a self-improving AI layer.

This is not a toy. Build everything with TypeScript strict mode, proper error handling, and production patterns. No mocks, no shortcuts.

Core stack: Node.js 20, Express.js, TypeScript, MongoDB (Mongoose), Redis, BullMQ, Playwright, Next.js 14, Tailwind CSS, shadcn/ui, NextAuth.js, Zod.

AI layer: Anthropic Claude API. Use `guardedAICall()` for EVERY Claude API call. Never call the Anthropic SDK directly — always go through the cost guard wrapper. Prompt templates are stored in MongoDB (promptTemplates collection) — never hardcode prompt text.

Start with Phase 1 (Section 15). Follow the folder structure in Section 14 exactly. Create the monorepo, install dependencies, configure TypeScript, set up MongoDB + Redis, create ALL Mongoose models from Section 04 (including the 5 new v3 collections: outcomeLogs, promptTemplates, scoringConfigs, selfImproveLogs, costLedger), and set up Express with auth middleware, rate limiting, and Zod validation.

Key constraints:

- AI gate (Section 06.1) must run before EVERY Claude API call
- All scoring weights loaded from DB — never hardcoded (Section 05)
- Outcome logging must be one click in the UI — feeds self-improvement loop (Section 08)
- Every scraping error must trigger the problem learner (Section 09)

Ask before making any architecture decision not covered in this document.

→ After Phase 1: paste Phase 2 tasks + reference Sections 10 (Scraping). After Phase 3: reference Section 06 in full when building the AI engine. After Phase 4: reference Sections 07-09 in full for the self-improvement layer.

UI DESIGN SYSTEM — Retro Brutalist Techno

This is not a generic SaaS dashboard. HydraFox looks like industrial surveillance software crossed with a Berlin techno flyer. Hard edges. Raw grid. Monospaced data. Orange bleeds on black. No gradients. No rounded corners. No friendliness. Claude Code must implement these rules across every component — no exceptions.

17.1 — Color System (CSS Variables — globals.css)

These are the ONLY colors used in the entire application. No Tailwind default colors. No external color decisions. Everything derives from these tokens.

```
/* globals.css — HydraFox Retro Brutalist Techno Color System */  
:root {  
  
    /* ■■ Core ████████████████████████████████████████████████████ */  
    --hf-black:          #080808;   /* true black — backgrounds */  
    --hf-black-2:        #111111;   /* card backgrounds */  
    --hf-black-3:        #1A1A1A;   /* elevated surfaces */  
    --hf-black-4:        #242424;   /* hover states, borders */  
  
    /* ■■ Orange Spectrum — the pulse of the system ██████████ */  
    --hf-orange:          #FF4D00;   /* primary accent — CTAs, badges, headers */  
    --hf-orange-dim:      #CC3D00;   /* pressed / active states */  
    --hf-orange-glow:     rgba(255, 77, 0, 0.15); /* glow overlays */  
    --hf-orange-ghost:    rgba(255, 77, 0, 0.06); /* subtle tints */  
    --hf-amber:           #FF8C00;   /* secondary accent — warnings, sub-labels */  
  
    /* ■■ Signal Colors — only for data states ████████████████ */  
    --hf-hot:             #FF4D00;   /* HOT leads — same as orange — intentional */  
    --hf-warm:            #FFD600;   /* WARM leads */  
    --hf-cold:            #4A90D9;   /* COLD leads */  
    --hf-dead:            #3A3A3A;   /* SKIP / inactive */  
    --hf-success:         #00C853;   /* positive outcomes */  
    --hf-error:           #FF1744;   /* errors, failures */  
    --hf-info:            #00B8D4;   /* informational */  
  
    /* ■■ Text ████████████████████████████████████████████████████ */  
    --hf-text-primary:    #F0F0F0;  
    --hf-text-secondary: #A0A0A0;  
    --hf-text-muted:      #555555;  
    --hf-text-code:       #A8FF78;   /* monospaced data readouts */  
  
    /* ■■ Borders ████████████████████████████████████████████████████ */  
    --hf-border:          #2A2A2A;  
    --hf-border-hot:      #FF4D00;   /* orange border for active/selected */  
    --hf-border-dim:      #1E1E1E;
```

[illegible]

17.2 — Typography System

Three font roles. No substitutions. Load all from Google Fonts or Fontsource npm packages.

Role	Font	Usage
Display / Headers	Space Grotesk Bold 700	All H1–H3, navigation labels, badge text, stat numbers
Body / UI	IBM Plex Mono Regular + Medium	ALL body text, descriptions, labels — this is the key brutalist choice: mono body text
Data Readouts	Share Tech Mono	Scores, IDs, timestamps, API keys, domain names — raw machine data

```

/* tailwind.config.ts – font registration */
fontFamily: {
  display: ["Space Grotesk", "sans-serif"],
  mono:    ["IBM Plex Mono", "monospace"], // used as DEFAULT body font
  data:    ["Share Tech Mono", "monospace"], // machine-data readouts
}

/* Typography scale – applied globally */
h1 { font: 700 32px/1.1 "Space Grotesk"; letter-spacing: -0.02em; color: var(--hf-orange); }
h2 { font: 700 20px/1.2 "Space Grotesk"; letter-spacing: -0.01em; color: var(--hf-text-primary); }
; }
h3 { font: 600 14px/1.3 "Space Grotesk"; letter-spacing: 0.04em; text-transform: uppercase; color: var(--hf-amber); }
body, p, td, li { font: 400 13px/1.6 "IBM Plex Mono"; color: var(--hf-text-secondary); }
.data-value { font: 400 12px/1.4 "Share Tech Mono"; color: var(--hf-text-code); }
.label      { font: 500 10px/1    "IBM Plex Mono"; letter-spacing: 0.1em; text-transform: uppercase; color: var(--hf-text-muted); }

```

17.3 — Layout System (Hard Grid — No Soft Curves)

The entire dashboard is built on a strict 8px grid. border-radius is 0px on EVERYTHING except pill badges (2px max). No shadows — use borders instead. No gradients — use flat fills + orange borders for hierarchy.

[illegible]

```
background-image:  
    linear-gradient(var(--hf-grid-line) 1px, transparent 1px),  
    linear-gradient(90deg, var(--hf-grid-line) 1px, transparent 1px);  
background-size: 32px 32px;  
}  
  
/* ■■ Topbar ████████████████████████████████████████████████████████████████ */  
.topbar {  
    grid-column: 1 / -1;  
    height: 48px;  
    background: var(--hf-black-2);  
    border-bottom: 1px solid var(--hf-border);  
    border-bottom-color: var(--hf-orange); /* orange underline — always */  
    display: flex; align-items: center; padding: 0 24px;  
}  
  
/* ■■ Sidebar ████████████████████████████████████████████████████████████████ */  
.sidebar {  
    background: var(--hf-black-2);  
    border-right: 1px solid var(--hf-border);  
    padding-top: 24px;  
}  
  
/* ■■ Card / Panel ████████████████████████████████████████████████████████████████ */  
.hf-card {  
    background: var(--hf-black-2);  
    border: 1px solid var(--hf-border);  
    padding: 16px 20px;  
    /* No shadow. Border does the work. */  
}  
  
.hf-card--active {  
    border-color: var(--hf-orange);  
    background: var(--hf-orange-ghost);  
}  
  
/* ■■ Scanline texture overlay (apply to .dashboard-shell) ■■ */  
.scanlines::after {  
    content: "";  
    position: fixed; inset: 0; pointer-events: none; z-index: 9999;  
    background: repeating-linear-gradient(  
        0deg, transparent, transparent 2px,  
        rgba(0,0,0,0.03) 2px, rgba(0,0,0,0.03) 4px  
    );  
}
```

17.4 — Component Specifications

Buttons

No radius. Flat fill. Orange border on primary. White text. Mono font.
Primary: bg=--hf-orange, text=black, border=none
Secondary: bg=transparent, text=--hf-orange, border=1px solid --hf-orange
Destructive: bg=transparent, text=--hf-error, border=1px solid --hf-error
Hover: brightness(1.15) + 1px border-offset glow: outline: 2px solid
var(--hf-orange-glow)
All uppercase. Letter-spacing: 0.08em. IBM Plex Mono Medium.

Priority Badges	HOT: bg=--hf-orange, text=black, font=Space Grotesk Bold WARM: bg=--hf-warm, text=black COLD: bg=--hf-cold, text=white SKIP: bg=--hf-dead, text=--hf-text-muted All caps. 10px. border-radius: 2px. Padding: 2px 8px. HOT badge pulses: animation: pulse-orange 2s ease-in-out infinite
Score Display	Large data number in Share Tech Mono 28px. Color mapped to priority. Below it: a thin 4px tall bar (full width of card) filled to score%, color matches priority. No circular progress – hard rectangular bar only.
Data Tables	border-collapse: collapse. Every row: border-bottom 1px --hf-border. Header row: border-bottom 2px --hf-orange. bg=--hf-black-3. Uppercase labels. 10px mono. Odd rows: bg=--hf-black-2. Even rows: bg=--hf-black-3. Selected row: border-left: 3px solid --hf-orange. bg=--hf-orange-ghost. No hover highlight – use cursor: crosshair on table rows.
Sidebar Nav	Width: 220px. Items: 44px tall. IBM Plex Mono 12px uppercase. Active: bg=--hf-orange-ghost, border-left: 3px solid --hf-orange, text=--hf-orange. Inactive: text=--hf-text-muted. Hover: text=--hf-text-primary, bg=--hf-black-3. Icons: Lucide icons, 16px, stroke-width=1.5. Tinted --hf-orange on active. Section dividers: 1px --hf-border with uppercase 9px label above.
Stat Cards	Top: label in 10px uppercase mono --hf-text-muted. Center: big number in Share Tech Mono 40px --hf-text-primary. Bottom: delta indicator (▲ +12% vs yesterday) in 11px mono, green/red. Left border: 3px solid --hf-orange on the active/key metric card. No icons in stat cards. Numbers only.
Form Inputs	bg=--hf-black-3. border=1px solid --hf-border. color=--hf-text-primary. Font: IBM Plex Mono 13px. Padding: 8px 12px. Focus: border-color=--hf-orange. No glow/shadow – hard border change only. Placeholder: --hf-text-muted. Error: border-color=--hf-error. Labels: 10px uppercase mono --hf-text-muted, 6px gap above input.
Code/Data Blocks	bg=--hf-black. border=1px solid --hf-border-dim. Font: Share Tech Mono 12px. Color: --hf-text-code (#A8FF78). Padding: 12px 16px. No syntax highlighting – single green color only. Left border: 2px solid --hf-orange for emphasis blocks.
Alerts & Callouts	No rounded corners. Left border 3px colored. bg matching color at 6% opacity. WARNING: border=--hf-error, bg=rgba(255,23,68,0.06) SUCCESS: border=--hf-success, bg=rgba(0,200,83,0.06) INFO: border=--hf-info, bg=rgba(0,184,212,0.06) Text: IBM Plex Mono 12px. Icon: Lucide 14px matching border color.
Job Progress Bar	4px tall flat bar. bg=--hf-black-3 (track). Fill=--hf-orange. Animated fill with CSS transition: width 0.3s ease. Active jobs: fill pulses using animation: pulse-width. No percentage label inside bar – show number separately in mono above.
Modal / Drawer	bg=--hf-black-2. border: 1px solid --hf-orange (top border only is orange, others --hf-border). Header: border-bottom 1px --hf-orange. bg=--hf-black-3. Title in Space Grotesk. Backdrop: rgba(0,0,0,0.85). No blur – hard dark overlay only.

17.5 — Animation System

Minimal animations. Every animation must have a purpose — loading state, data refresh, status change. No decorative animations. No parallax. No page transitions with slides. The system feels alive but controlled — like a terminal, not a marketing site.

```
/* animations.css — HydraFox approved animations only */

/* 1. HOT badge pulse — draws attention to priority leads */
@keyframes pulse-orange {
  0%, 100% { box-shadow: 0 0 0 rgba(255,77,0,0.4); }
  50%      { box-shadow: 0 0 6px rgba(255,77,0,0); }
}
.badge-hot { animation: pulse-orange 2s ease-in-out infinite; }

/* 2. Score bar fill on mount — data loads visibly */
@keyframes fill-bar {
  from { width: 0%; }
  to   { width: var(--score-pct); }
}
.score-bar-fill { animation: fill-bar 0.6s cubic-bezier(0.16,1,0.3,1) forwards; }

/* 3. New lead row flash — when a scraping job drops new leads */
@keyframes row-appear {
  0%   { background: rgba(255,77,0,0.2); border-left-color: var(--hf-orange); }
  100% { background: transparent; }
}
.lead-row--new { animation: row-appear 1.5s ease-out forwards; }

/* 4. Blink cursor — on active job status labels */
@keyframes blink {
  0%, 100% { opacity: 1; } 50% { opacity: 0; }
}
.status-active::after {
  content: "_"; animation: blink 1s step-end infinite;
  color: var(--hf-orange); margin-left: 4px;
}

/* 5. Scanline flicker on page load — one time only */
@keyframes flicker {
  0% { opacity: 0.9; } 5% { opacity: 1; } 10% { opacity: 0.85; }
  15% { opacity: 1; } 100% { opacity: 1; }
}
.dashboard-shell { animation: flicker 0.3s ease-in forwards; }

/* 6. Number count-up — stats on dashboard mount */
/* Use CountUp.js library — no CSS for this one */
/* Config: duration: 1.2s, separator: ",", decimal: "." */

/* FORBIDDEN animations (do not use):
   - fade-in on scroll      - slide-in transitions      - hover scale/zoom
   - rotation effects      - gradient shifts            - parallax */
```

17.6 — Page-by-Page UI Specifications

Every page has a specific layout and component arrangement. Claude Code must implement each page exactly as specified here.

/dashboard

TOPBAR: Logo (Space Grotesk Bold 18px 'HYDRAFOX' in --hf-orange) + breadcrumb 'DASHBOARD' + right-side: AI spend today badge + user avatar.

STAT ROW (4 cards, equal width): TOTAL LEADS / HOT TODAY / OUTREACH SENT / AI SPEND \$.
Key metric card (HOT TODAY) gets left border --hf-orange.

MAIN GRID (2 cols, 60/40 split):

Left: 7-day lead volume chart. Recharts LineChart. Line color: --hf-orange. No area fill. Grid lines: --hf-border. Axis labels: Share Tech Mono 10px --hf-text-muted.

Right: Job Status panel. Each active job: name / progress bar / status label with blink cursor.

BOTTOM: Recent Leads table (last 10). Columns: BUSINESS / SCORE / PRIORITY / STATUS / SOURCE / TIME.
Self-improvement status strip: last run time + '3 WEIGHTS UPDATED' or 'NO CHANGES' in mono.

/leads

TOPBAR + filter bar (full width below topbar): Score range slider (--hf-orange thumb) / Priority dropdown / Status multi-select / Industry select / Source select / Search input with magnifier icon.

TABLE occupies full remaining height. Sticky header.

Columns: [] checkbox / BUSINESS (mono) / INDUSTRY / SCORE bar+number / PRIORITY badge / STATUS / SOURCE / DATE.

SCORE column: inline mini bar (80px wide, 4px tall) + Share Tech Mono number.

Row click → navigate to /leads/[id].

Bulk action bar appears above table when rows selected: 'N SELECTED — [EXPORT CSV] [UPDATE STATUS■] [ARCHIVE]' — all caps mono buttons.

/leads/[id]

Full-width header card: BUSINESS NAME (32px Space Grotesk) on left. Right side: SCORE (64px Share Tech Mono colored by priority) + Priority badge + Status selector.

Below header: 3px full-width orange bar always.

TWO COLUMN LAYOUT (55/45):

LEFT COLUMN:

— Business Details card: domain (mono link) / industry / location / source tag.

— Enrichment Signals card: tech stack pills (flat, --hf-border outlined) / performance bars / automation level indicator / social activity indicator.

— Score Breakdown card: factor list with per-factor point values in Share Tech Mono.

Positive factors: --hf-success. Negative: --hf-error.

RIGHT COLUMN:

— AI Analysis card: primaryPain in large body text. pains list. Confidence score.

— Contacts card: contact name/role/email/linkedin in rows.

— Outreach panel: pitch angle quote block + [GENERATE MESSAGE] button.

Generated message appears below in editable textarea (mono font).

OUTCOME BAR (sticky at bottom, full width): [REPLIED] [INTERESTED] [CONVERTED] [IGNORED] [BOUNCED] — one-click outcome logging. Large buttons. Most-used action (REPLIED) gets --hf-orange.

/self-improve

PAGE TITLE: 'SELF-IMPROVEMENT ENGINE' in Space Grotesk 28px --hf-orange.

Subtitle: last run timestamp in Share Tech Mono.

TOP ROW (3 stat cards): OUTCOMES LOGGED / WEIGHTS CHANGED / PROMPTS UPDATED.

SECTION: SCORING WEIGHTS — table showing all factors, current weight, last change (+/- delta in --hf-success/--hf-error), confidence level.

Pending changes flagged with REVIEW badge in --hf-amber.

[APPROVE] / [REJECT] buttons per pending row.

SECTION: PROMPT PERFORMANCE — table: template name / version / reply rate / times used / status badge (ACTIVE/TESTING/RETIRED).

Bar sparkline per row showing reply rate trend (last 7 data points).

SECTION: PROBLEM LOG — chronological log of operational problems and auto-fixes. Each row: timestamp (mono) / type badge / problem description / action taken.

CRITICAL items get --hf-error left border. WARNING: --hf-amber.

/jobs

JOB STATUS DASHBOARD — terminal aesthetic.

Active jobs section: each job is a row with blinking status cursor.

FORMAT: [JOB_ID in mono] [QUEUE NAME] [■■■■■■■■■ 62%] [STARTED 4m ago] [PAUSE] [KILL]

Progress bar: orange fill, black track, 8px tall, sharp edges.

QUEUE STATS row: 4 numbers — WAITING / ACTIVE / COMPLETED TODAY / FAILED TODAY.

Failed jobs section: red left border. Error message in mono. [RETRY] button.

NEW JOB form at bottom or in slide-out drawer:

Source: [GOOGLE MAPS ■] Location: [] Industry: [] Limit: [50]

[QUEUE JOB] button — full-width, --hf-orange bg, black text.

17.7 — Tailwind Configuration

Extend Tailwind with HydraFox tokens. Never use default Tailwind colors in components. Always use the hf-* custom tokens defined below.

```
// tailwind.config.ts
import type { Config } from "tailwindcss";
export default {
  content: ["/app/**/*.ts,tsx", "/components/**/*.ts,tsx"],
  theme: {
    borderRadius: { none:"0", sm:"2px", DEFAULT:"0" }, // force no radius everywhere
    extend: {
      fontFamily: {
        display: ["Space Grotesk","sans-serif"],
        mono: ["IBM Plex Mono","monospace"],
        data: ["Share Tech Mono","monospace"],
      },
      colors: {
        hf: {
          black: "#080808", "black-2": "#111111", "black-3": "#1A1A1A", "black-4": "#242424",
          orange: "#FF4D00", "orange-dim": "#CC3D00",
          amber: "#FF8C00",
          hot: "#FF4D00", warm:"#FFD600", cold:"#4A90D9", dead:"#3A3A3A",
        }
      }
    }
  }
}
```

```

    success: "#00C853", error: "#FF1744", info: "#00B8D4",
    "text-primary": "#F0F0F0", "text-secondary": "#A0A0A0",
    "text-muted": "#555555", "text-code": "#A8FF78",
    border: "#2A2A2A", "border-hot": "#FF4D00",
  },
},
boxShadow: { none: "none" }, // no shadows system-wide
dropShadow: { none: "none" },
backgroundImage: {
  "hf-grid": "linear-gradient(rgba(255,77,0,0.08) 1px, transparent 1px), linear-gradient(
90deg, rgba(255,77,0,0.08) 1px, transparent 1px)",
},
backgroundSize: { "hf-grid": "32px 32px" },
keyframes: {
  "pulse-orange": {
    "0%,100%": { boxShadow: "0 0 0 rgba(255,77,0,0.4)" },
    "50%": { boxShadow: "0 0 0 6px rgba(255,77,0,0)" },
  },
  "fill-bar": { from: {width: "0%"}, to: {width: "var(--score-pct)"} },
  "row-appear": {
    "0%": { background: "rgba(255,77,0,0.2)" },
    "100%": { background: "transparent" },
  },
  "blink": { "0%,100%": {opacity: "1"}, "50%": {opacity: "0"} },
  "flicker": {
    "0%": {opacity: "0.9"}, "5%": {opacity: "1"}, "10%": {opacity: "0.85"}, "100%": {opacity: "1"}
  },
},
},
animation: {
  "pulse-orange": "pulse-orange 2s ease-in-out infinite",
  "fill-bar": "fill-bar 0.6s cubic-bezier(0.16,1,0.3,1) forwards",
  "row-appear": "row-appear 1.5s ease-out forwards",
  "blink": "blink 1s step-end infinite",
  "flicker": "flicker 0.3s ease-in forwards",
},
},
},
plugins: [],
} satisfies Config;

```

17.8 — Absolute Design Rules (Never Break These)

- ✓ **ALWAYS use border-radius: 0** — Everything is sharp. No softness anywhere.
- ✓ **ALWAYS use IBM Plex Mono for body text** — Not Inter. Not system-ui. Mono body is the identity.
- ✓ **ALWAYS show score as a flat horizontal bar** — Never circular. Never donut. Hard rectangle.
- ✓ **ALWAYS use uppercase labels** — Section labels, nav items, column headers — all caps, letter-spaced.
- ✓ **ALWAYS log every interaction** — Timestamps visible on all actions in Share Tech Mono.
- ✓ **ALWAYS use orange for the single most important CTA on any screen** — One orange element owns each page.
- ✗ **NEVER use any color not in the --hf-* token system** — No Tailwind blue-500, no hex literals in components.
- ✗ **NEVER use box-shadow** — Use border instead to create elevation/depth.
- ✗ **NEVER add gradients** — Flat fills only. The grid background is the only texture.

- ✗ **NEVER round corners** — border-radius: 0 on everything except 2px on pill badges only.
- ✗ **NEVER use decorative icons as page heroes** — Icons are 16px utility elements only.
- ✗ **NEVER use loading spinners** — Use progress bars or blinking status text. No spinning circles.
- ✗ **NEVER use toast notifications that slide in from corners** — Use inline status updates in the data table row.
- ✗ **NEVER put the HOT badge in any color other than --hf-orange** — It owns that color. No exceptions.

◆ Claude Code implementation note: Before building any UI component, ask: does this look like it belongs in a Berlin techno venue's surveillance system? If yes, proceed. If it looks like a SaaS startup's dashboard, rebuild it.

17.9 — Reference Component: Lead Score Card

This component embodies the entire design system. Claude Code must use this as the reference for all component decisions.

```
// components/leads/ScoreCard.tsx
import { cn } from '@lib/utils'

const PRIORITY_CONFIG = {
  hot: { color:'text-hf-hot', bar:'bg-hf-hot', border:'border-hf-orange', label:'HOT', pulse:true },
  warm: { color:'text-hf-warm', bar:'bg-hf-warm', border:'border-hf-amber', label:'WARM', pulse:false },
  cold: { color:'text-hf-cold', bar:'bg-hf-cold', border:'border-hf-cold', label:'COLD', pulse:false },
  skip: { color:'text-hf-dead', bar:'bg-hf-dead', border:'border-hf-dead', label:'SKIP', pulse:false },
} as const

export function ScoreCard({ lead }: { lead: Lead }) {
  const cfg = PRIORITY_CONFIG[lead.priority]
  // Outer card: sharp edges, left border color = priority
  // className: 'hf-card border border-hf-border bg-hf-black-2 border-l-[3px]' + cfg.border

  // HEADER ROW: business name left | score number + priority badge right
  // SCORE: font-data text-4xl colored by cfg.color (Share Tech Mono)
  // BADGE: 10px Space Grotesk Bold ALL CAPS, bg=priority color, text=black
  // HOT badge gets: animate-pulse-orange

  // SCORE BAR (below header):
  // 4px tall flat rectangle. Track: bg-hf-black-4
  // Fill div: className=cfg.bar + 'animate-fill-bar'
  // style={{ '--score-pct': lead.score + '%' } as CSSProperties}

  // TECH STACK PILLS:
  // techStack.slice(0,4) - font-data 10px, border border-hf-border, no bg

  // PRIMARY PAIN QUOTE:
  // font-mono text-xs text-hf-text-secondary
  // border-l-2 border-hf-orange pl-3 (orange left bar = quote block)

  // FOOTER ROW:
  // border-t border-hf-border pt-3 mt-3
  // left: SOURCE · INDUSTRY in font-data 10px uppercase
  // right: creation date in font-data 10px
}
```

END OF BUILD DOCUMENT — v3.0

HydraFox Lead Engine — AI Optimized + Self-Improving + Retro Brutalist Techno UI — Internal Use Only